二维浅水方程 4D-Var 离散伴随说明

原版程序的历史说明（见 `baseline/original.c`）

杨鹏亮

2026 年 10 月 24 日

版本说明（**2026-10-16**）：本文保留为原版程序的推导和实验记录，不描述现行实现。现行代码使用检查点重算伴随、相关背景协方差和白化控制变量，以 SciPy L-BFGS-B 优化；配置、验证和新实验结果见 `README.md`、`RESULTS.md` 与 `var_output/summary.json`。

摘要

本文只记录当前程序 `swe2d_4dvar.c` 实际实现的离散伴随。前向模式是二维浅水方程的有限体积离散，界面通量为 Rusanov 通量，时间推进为两次 Euler 子步组成的 RK2。4D-Var 控制变量是初始全状态，观测只抽取若干网格点上的水深。梯度由完整前向轨道、逐步反向的离散伴随和背景项共同组成。本文不再推导连续伴随 PDE，因为代码没有使用连续伴随后再离散的路线。

目录

1 状态、网格和代价函数

单元状态为守恒变量

$$\mathbf{q}_{i,j} = \begin{pmatrix} h_{i,j} \\ (hu)_{i,j} \\ (hv)_{i,j} \end{pmatrix}.$$

代码中一个完整场用三个数组 `h`, `hu`, `hv` 保存，单元索引为

$$k = i + N_x j.$$

当前 4D-Var 只优化初始场 $\mathbf{Q}^0 \in \mathbb{R}^{3N}$。背景场记为 $\mathbf{Q}_b$，观测位于每个同化窗口末端。代价函数为

$$J(\mathbf{Q}^0) = \frac{1}{2}(\mathbf{Q}^0 - \mathbf{Q}_b)^T B^{-1}(\mathbf{Q}^0 - \mathbf{Q}_b) + \frac{1}{2}\sum_{c=1}^{N_c} \left(H\mathbf{Q}^{n_c} - \mathbf{y}_c\right)^T R^{-1} \left(H\mathbf{Q}^{n_c} - \mathbf{y}_c\right). \quad (1)$$

这里 $N = N_x N_y$，$n_c = c\,$`VAR_STEPS_WINDOW`。H 是观测算子，从全状态中抽取观测单元的水深 h 分量：

$$H\mathbf{Q}^{n_c} = \left(h_{o_1}^{n_c}, \ldots, h_{o_{N_{\mathrm{obs}}}}^{n_c}\right)^T,$$

其中 o_p 是第 p 个观测单元索引。

矩阵 B 是背景误差协方差矩阵；当前代码采用逐格点、逐变量独立的对角形式

$$B = \mathrm{diag}\left(\sigma_h^2 I_N,\, \sigma_m^2 I_N,\, \sigma_m^2 I_N\right).$$

矩阵 R 是观测误差协方差矩阵；当前代码假设所有水深观测误差相互独立且方差相同，

$$R = \sigma_o^2 I_{N_{\mathrm{obs}}}.$$

因此式 (1) 展开后正是代码中 `evaluate_cost_gradient()` 计算的逐分量加权平方和。

2 前向离散模式

2.1 物理通量

令 $m = hu$，$n = hv$。物理通量为

$$\mathbf{f}(\mathbf{q}) = \begin{pmatrix} m \\ m^2/h + \frac{1}{2}gh^2 \\ mn/h \end{pmatrix}, \qquad \mathbf{g}(\mathbf{q}) = \begin{pmatrix} n \\ mn/h \\ n^2/h + \frac{1}{2}gh^2 \end{pmatrix}. \tag{2}$$

代码在分母中使用 `safe_h()`，即 h 小于 `HMIN` 时用 `HMIN` 代替。

2.2 Rusanov 通量

x 方向界面通量为

$$\widehat{\mathbf{f}}(\mathbf{q}_L, \mathbf{q}_R) = \frac{1}{2}\left[\mathbf{f}(\mathbf{q}_L) + \mathbf{f}(\mathbf{q}_R)\right] - \frac{1}{2}s(\mathbf{q}_R - \mathbf{q}_L), \tag{3}$$

其中

$$s = \max(a_L, a_R), \qquad a(\mathbf{q}) = |u| + \sqrt{gh}, \qquad u = m/h.$$

y 方向完全类似，只是速度用 $v = n/h$：

$$a_y(\mathbf{q}) = |v| + \sqrt{gh}.$$

这些式子对应 `rusanov_x()` 和 `rusanov_y()`。

2.3 Euler 子步

`euler_update()` 实现

$$\mathbf{q}_{i,j}^{+} = \mathbf{q}_{i,j} - \frac{\Delta t}{\Delta x}\left(\widehat{\mathbf{f}}_{i+1/2,j} - \widehat{\mathbf{f}}_{i-1/2,j}\right) - \frac{\Delta t}{\Delta y}\left(\widehat{\mathbf{g}}_{i,j+1/2} - \widehat{\mathbf{g}}_{i,j-1/2}\right). \tag{4}$$

越界邻居由 `get_state_bc()` 钳制到最近的边界单元，因此边界是离散零梯度延拓。

2.4 RK2 步

`step_rk2_store()` 使用

$$\mathbf{Q}^{(1)} = E(\mathbf{Q}^{n}), \tag{5}$$

$$\mathbf{Q}^{(2)} = E(\mathbf{Q}^{(1)}), \tag{6}$$

$$\mathbf{Q}^{n+1} = \frac{1}{2}\mathbf{Q}^{n} + \frac{1}{2}\mathbf{Q}^{(2)}. \tag{7}$$

其中 E 是式 (4) 的 Euler 子步。代码保存 `traj[n]` 和 `stage1[n]`，分别对应 $\mathbf{Q}^{n}$ 和 $\mathbf{Q}^{(1)}$，供离散伴随反向使用。

3 界面通量的 Jacobian

离散伴随需要 Rusanov 通量对两侧状态的导数。物理通量 Jacobian 为

$$A(\mathbf{q}) = \frac{\partial \mathbf{f}}{\partial \mathbf{q}} = \begin{pmatrix} 0 & 1 & 0 \\ gh - m^2/h^2 & 2m/h & 0 \\ -mn/h^2 & n/h & m/h \end{pmatrix}, \tag{8}$$

$$B(\mathbf{q}) = \frac{\partial \mathbf{g}}{\partial \mathbf{q}} = \begin{pmatrix} 0 & 0 & 1 \\ -mn/h^2 & n/h & m/h \\ gh - n^2/h^2 & 0 & 2n/h \end{pmatrix}. \tag{9}$$

代码中的 `flux_jacobian_x()` 和 `flux_jacobian_y()` 还包含 `safe_h()` 分支：当原始 $h \le$ `HMIN` 时，关于 h 的导数被置零，以匹配前向分支。

对 x 方向 Rusanov 通量，定义

$$\Delta\mathbf{q} = \mathbf{q}_R - \mathbf{q}_L.$$

若 $a_L \ge a_R$，则 $s = a_L$，$\nabla_{\mathbf{q}_L} s = \nabla a_L$，$\nabla_{\mathbf{q}_R} s = 0$；否则相反。于是

$$\widehat{F}_L = \frac{1}{2}A(\mathbf{q}_L) + \frac{1}{2}sI - \frac{1}{2}\Delta\mathbf{q}\,(\nabla_{\mathbf{q}_L} s)^T, \tag{10}$$

$$\widehat{F}_R = \frac{1}{2}A(\mathbf{q}_R) - \frac{1}{2}sI - \frac{1}{2}\Delta\mathbf{q}\,(\nabla_{\mathbf{q}_R} s)^T. \tag{11}$$

这正是 `rusanov_x_jacobians()` 的计算。y 方向的 `rusanov_y_jacobians()` 用同样公式，只把左右状态换成 bottom/top 状态，并用 $B(\mathbf{q})$ 和 a_y。

速度导数在代码中为

$$\nabla a_x = \begin{pmatrix} -\operatorname{sgn}(u)m/h^2 + \frac{1}{2}g/\sqrt{gh} \\ \operatorname{sgn}(u)/h \\ 0 \end{pmatrix}, \qquad \nabla a_y = \begin{pmatrix} -\operatorname{sgn}(v)n/h^2 + \frac{1}{2}g/\sqrt{gh} \\ 0 \\ \operatorname{sgn}(v)/h \end{pmatrix}. \tag{12}$$

在 $u = 0$ 或 $v = 0$ 时，`signum()` 返回零。

4 Euler 子步的离散伴随

设某个输出单元 (i, j) 的伴随为

$$\boldsymbol{\lambda}^+_{i,j} = \begin{pmatrix} \lambda^+_h \\ \lambda^+_{hu} \\ \lambda^+_{hv} \end{pmatrix}.$$

`euler_adjoint()` 对每个输出单元做反向累加。首先，前向 Euler 子步中有恒等项 $\mathbf{q}^+_{i,j} \leftarrow \mathbf{q}_{i,j}$，因此

$$\boldsymbol{\lambda}_{i,j} \mathrel{+}= \boldsymbol{\lambda}^+_{i,j}.$$

然后四个界面按前向式 (4) 中的符号回传：

$$\text{右界面:}\quad \boldsymbol{\lambda}_{i,j} \mathrel{+}= -\frac{\Delta t}{\Delta x}\widehat{F}_L^T\boldsymbol{\lambda}^+_{i,j}, \qquad \boldsymbol{\lambda}_{i+1,j} \mathrel{+}= -\frac{\Delta t}{\Delta x}\widehat{F}_R^T\boldsymbol{\lambda}^+_{i,j}, \tag{13}$$

$$\text{左界面:}\quad \boldsymbol{\lambda}_{i-1,j} \mathrel{+}= \frac{\Delta t}{\Delta x}\widehat{F}_L^T\boldsymbol{\lambda}^+_{i,j}, \qquad \boldsymbol{\lambda}_{i,j} \mathrel{+}= \frac{\Delta t}{\Delta x}\widehat{F}_R^T\boldsymbol{\lambda}^+_{i,j}, \tag{14}$$

$$\text{上界面:}\quad \boldsymbol{\lambda}_{i,j} \mathrel{+}= -\frac{\Delta t}{\Delta y}\widehat{G}_B^T\boldsymbol{\lambda}^+_{i,j}, \qquad \boldsymbol{\lambda}_{i,j+1} \mathrel{+}= -\frac{\Delta t}{\Delta y}\widehat{G}_T^T\boldsymbol{\lambda}^+_{i,j}, \tag{15}$$

$$\text{下界面:}\quad \boldsymbol{\lambda}_{i,j-1} \mathrel{+}= \frac{\Delta t}{\Delta y}\widehat{G}_B^T\boldsymbol{\lambda}^+_{i,j}, \qquad \boldsymbol{\lambda}_{i,j} \mathrel{+}= \frac{\Delta t}{\Delta y}\widehat{G}_T^T\boldsymbol{\lambda}^+_{i,j}. \tag{16}$$

矩阵转置乘向量由 `add_matT_vec_to_cell()` 完成。

边界处使用和前向相同的钳制索引。例如 $i = 0$ 时，左邻居仍然是 $i = 0$；因此边界单元会收到来自自身和钳制 ghost 路径的累加贡献。这不是连续边界条件的近似，而是前向离散索引映射的精确转置。

5 RK2 步的离散伴随

由式 (7) 可得反向顺序：

$$\mathbf{\Lambda}^{(2)} = \frac{1}{2}\mathbf{\Lambda}^{n+1}, \tag{17}$$

$$\mathbf{\Lambda}^{n} = \frac{1}{2}\mathbf{\Lambda}^{n+1}, \tag{18}$$

$$\mathbf{\Lambda}^{(1)} = E'(\mathbf{Q}^{(1)})^T\mathbf{\Lambda}^{(2)}, \tag{19}$$

$$\mathbf{\Lambda}^{n} \mathrel{+}= E'(\mathbf{Q}^{n})^T\mathbf{\Lambda}^{(1)}. \tag{20}$$

这对应 `rk2_adjoint_step()`：

- `lam_stage2` 接收 $\frac{1}{2}\mathbf{\Lambda}^{n+1}$；
- `lam_n` 先接收 $\frac{1}{2}\mathbf{\Lambda}^{n+1}$；
- 在 `Q1` 上调用一次 `euler_adjoint()`；
- 在 `Qn` 上再调用一次 `euler_adjoint()`；
- 最后把第二次反传结果累加到 `lam_n`。

6 观测项和全窗口梯度

观测算子只抽取水深：

$$H\mathbf{Q}^{n_c} = \left(h_{o_1}^{n_c}, \ldots, h_{o_{N_{\mathrm{obs}}}}^{n_c}\right)^T.$$

因此在观测时刻只向 h 分量注入

$$\lambda_h(o_p) \mathrel{+}= \frac{h_{o_p}^{n_c} - y_{c,p}}{\sigma_o^2}. \tag{21}$$

这对应 `obs_gradient_add()`。

`evaluate_cost_gradient()` 的顺序是：

1. 从当前初值 `x0` 前向积分，保存 `traj` 和 `stage1`；
2. 计算背景项，并把背景项梯度写入 `grad`；
3. 遍历所有观测时刻，累计观测代价；
4. 从末时刻反向到初始时刻；若当前时间层是观测时刻，先执行式 (21)，再反传一个 RK2 步；
5. 把回到 $t = 0$ 的伴随 `lam_curr` 加到背景项梯度上。

最终梯度为

$$\nabla_{\mathbf{Q}^0} J = B^{-1}(\mathbf{Q}^0 - \mathbf{Q}_b) + \mathbf{\Lambda}^0, \tag{22}$$

其中 B 是对角矩阵，每个单元对应

$$\mathrm{diag}(\sigma_h^2, \sigma_m^2, \sigma_m^2).$$

7 结论

当前程序实现的是离散伴随，不是连续伴随 PDE 的再离散。保持一致的关键是：

1. 使用和前向完全相同的 Rusanov 分支、边界钳制和固定时间步长；
2. 对 Euler 子步逐界面累加 Jacobian 转置贡献；
3. 对 RK2 严格按组合映射反向传播；
4. 在观测时刻只向被观测水深分量注入 $R^{-1}(H\mathbf{Q} - \mathbf{y})$；
5. 最后把初始伴随加到背景项梯度上。

这些步骤与 `swe2d_4dvar.c` 中的函数调用顺序一一对应。